
Constraint Tax in Open-Weight LLMs: An Empirical Study of Tool Calling Suppression Under Structured Output Constraints

Fangzheng Li^{1,2,†}

¹Focus AI Center, Focus Technology Co., Ltd.

²Nanjing University of Science and Technology

lifangzheng@focuschina.com

Fangzheng_Li@njjust.edu.cn

Aimin Zhang^{1,†*}

¹Focus AI Center, Focus Technology Co., Ltd.

zhangaimin@focuschina.com

Chen Lv¹

¹Focus AI Center, Focus Technology Co., Ltd.

lvchen1018@focuschina.com

Abstract

Tool Calling and Structured Output are two core capabilities of modern Agent systems. However, the behavioral interaction effects when both are simultaneously enabled during the same inference process remain understudied. This paper reports and investigates a phenomenon observed in a production Agent system: when Tool Calling and JSON Schema constraints are simultaneously enabled, the tool invocation rate of multiple open-weight models drops from nearly 100% to 0%, while JSON format compliance remains close to 100%. We define this phenomenon as Tool Suppression and regard it as a behavioral-level manifestation of the Constraint Tax in Agent execution scenarios.

To systematically analyze this issue, we conduct controlled experiments on 7 model instances, covering parameter scales from 35B to 397B, multiple model families, two inference frameworks, and both local and cloud deployment environments. Experimental results demonstrate that Tool Suppression is stably reproduced across all tested open-weight models and does not alleviate with increasing model scale. Further diagnostic experiments reveal that models can still identify tool requirements and complete tool planning, but abandon actual tool execution during the action selection phase. Based on these findings, we propose the Constraint Priority Inversion (CPI) hypothesis to explain the overriding effect of structured output constraints on tool execution decisions.

To address this problem, this paper further proposes the Transparent Two-Pass Execution framework, which successfully restores tool calling behavior while maintaining structured output capability by decoupling tool execution from structured output generation in the temporal dimension. Our research demonstrates that the impact of the Constraint Tax manifests not only in answer quality degradation but may further evolve into action selection failure in Agent systems.

Keywords Constraint Tax · Tool Suppression · Constraint Priority Inversion · Large Language Models · Agent Systems

1 Introduction

1.1 Tool-Augmented LLM Agents

As Large Language Models (LLMs) evolve from dialogue systems to Agents with autonomous execution capabilities, Tool Augmentation has become a core component of modern Agent architectures. By invoking

*Corresponding author: zhangaimin@focuschina.com

external tools such as search engines, databases, code interpreters, and enterprise internal systems, LLMs can overcome the timeliness and coverage limitations of parametric knowledge, thereby completing complex tasks that require real-time information acquisition and environmental interaction [2, 3, 10].

Existing Agent frameworks commonly adopt the “Reasoning-Acting” paradigm. Representative works such as ReAct [2] achieve task solving through alternating execution of thinking and action steps; Toolformer [3] further demonstrates that language models can learn to autonomously invoke external tools at appropriate times. With the development of standardized protocols such as Function Calling interfaces and MCP (Model Context Protocol), tool calling capability has gradually become infrastructure for production-level Agent systems.

Meanwhile, Structured Output is emerging as another critical capability in Agent deployment. Unlike traditional natural language responses, Agents in production environments typically need to output JSON objects according to predefined Schemas for reliable parsing and invocation by frontend components, automated workflows, API services, and multi-Agent systems. Therefore, in actual deployment scenarios, tool calling and structured output are often not independent capabilities but consecutive stages in the same execution pipeline: the Agent first acquires external information through tools, then organizes the results into structured responses satisfying Schema constraints.

However, existing research typically evaluates tool calling capability and structured output capability separately. There remains a lack of systematic empirical research on the behavioral interaction effects that may arise when both are simultaneously enabled during the same Agent execution process.

1.2 Constraint Tax and an Unexplored Agent Failure Mode

Recent research has begun to focus on the side effects that structured output constraints impose on language model behavior. Ray [1] proposed the Constraint Tax concept to describe the performance cost that models pay when satisfying structured output requirements. Their experiments demonstrate that imposing strict Schema constraints on small language models significantly reduces answer accuracy while increasing the proportion of “format-correct but content-incorrect” outputs. This finding indicates that structured output constraints are not behaviorally neutral engineering mechanisms but may alter the model’s resource allocation and decision-making strategies during generation.

However, existing research on the Constraint Tax almost exclusively focuses on text generation tasks, with primary attention to answer quality degradation, reasoning capability decline, and accuracy loss due to format constraints. In these scenarios, the model’s core task remains text generation, and structured output constraints only affect the expression form of the final answer.

Agent systems introduce a fundamentally different execution paradigm. In typical Agent workflows, models not only need to generate answers but also need to decide whether to invoke external tools, when to invoke tools, and how to utilize the information returned by tools to complete tasks. Therefore, when structured output constraints and tool calling capability coexist, the object of constraint influence is no longer merely the final text generation result but may further act on the Agent’s action selection process.

This leads to a question that has not been systematically studied:

When structured output constraints are imposed on Agent execution pipelines requiring tool calling, does the Constraint Tax still only manifest as answer quality degradation, or does it further affect the Agent’s action selection behavior?

The core finding of this paper indicates that the latter case indeed exists. In our experiments, models do not simply produce lower-quality answers but directly abandon the tool execution process under joint constraint conditions. In other words, the impact of the Constraint Tax escalates from “answering incorrectly” to “action not executed,” thereby forming a previously undescribed Agent failure mode.

1.3 Observation from a Production Agent System

This research did not originate from theoretical hypotheses but from an anomalous phenomenon observed when deploying an Agent system in a real production environment. The research object is a tool-augmented Agent for foreign trade inquiry analysis tasks, whose typical execution pipeline includes three stages: external information retrieval, content analysis, and structured result generation. Since the system requires automated integration with downstream components, all final outputs must satisfy predefined JSON Schema constraints.

During system launch testing, we discovered that the Agent occasionally generated analysis results with obviously incomplete content or lacking factual basis. Preliminary inspection showed that the output format consistently met Schema requirements, so the problem did not originate from JSON parsing failures or format errors. However, upon further tracking of Agent execution logs, we observed an even more anomalous phenomenon: some requests never triggered any tool calling behavior before generating structured responses.

This result clearly contradicted our expectations. According to the Agent design logic, when tasks involve enterprise background investigation, market information collection, or product knowledge supplementation, the model should first invoke search tools or knowledge base tools to acquire external information, then generate analysis conclusions based on retrieval results. In fact, under conditions without enabling structured output constraints, the same model can stably complete this execution pipeline; and when only testing JSON Schema constraints, the model also maintains high format compliance rates. Therefore, from a capability perspective, the model possesses both tool calling capability and structured output capability.

What was truly surprising was that both capabilities performed normally when tested independently, but exhibited systematic failure when simultaneously enabled in the same request. The model no longer executed tool calls but directly generated structured results meeting Schema requirements, with some content appearing as null values, default values, or unverified speculative information. This phenomenon did not occur randomly but was stably reproduced in subsequent repeated experiments, ultimately prompting us to conduct the systematic research presented in this paper.

This observation constitutes the starting point of this paper’s research question: why do two capabilities that work normally independently lead to the Agent abandoning tool execution behavior when jointly enabled?

1.4 Contributions

This paper aims to answer a question that has not been systematically studied: whether structured output constraints affect an Agent’s tool execution behavior. Addressing this question, we make the following contributions:

- Discovery and formal definition of a new Agent failure mode. We identify and define the Tool Suppression phenomenon: when tool calling and structured output constraints are simultaneously enabled, open-weight models systematically abandon tool execution behavior, even though their tool calling capability and structured output capability remain normal under independent conditions. This discovery extends Constraint Tax research from answer quality degradation to the Agent action selection level.
- Verification of its universality through cross-model controlled experiments. We conduct systematic experiments on 7 model instances, covering parameter scales from 35B to 397B, multiple model families, two mainstream inference frameworks, and both local and cloud deployment environments. Experimental results demonstrate that Tool Suppression can be stably reproduced and does not alleviate with increasing model scale.
- Proposal of the Constraint Priority Inversion (CPI) hypothesis. Based on diagnostic experiment results, we find that models can typically correctly identify tool requirements and complete tool planning but abandon tool execution during the final action selection phase. To explain this phenomenon, this paper proposes the CPI mechanism, arguing that structured output constraints override tool execution priority during the decision process, leading to behavioral-level failure.
- Proposal and validation of Transparent Two-Pass Execution. To address the Tool Suppression problem, we design a framework-level mitigation solution that does not require modifying business logic. By decoupling tool execution from structured output generation in the temporal dimension, it successfully restores tool calling behavior while maintaining structured output capability.
- Establishment of a Tool Suppression behavioral taxonomy. Based on experimental observations, we further propose five categories of Tool Suppression behavior (TS-A to TS-E) to characterize typical failure modes of different models under joint constraint scenarios, providing a unified analytical framework for subsequent research.

2 Background

2.1 Tool Calling in LLM Agents

Tool Calling refers to the ability of language models to interact with the external environment by generating structured function call requests. Unlike traditional language models that rely solely on parametric knowledge, tool-augmented Agents can access external resources such as search engines, databases, code interpreters, and enterprise internal systems, thereby completing tasks that require real-time information acquisition, environmental perception, or external execution [2, 3, 10]. Therefore, tool calling has become one of the important features distinguishing modern Agent systems from pure dialogue models.

Existing Agent frameworks commonly treat tool calling as an execution process composed of multiple stages, including Task Understanding, Tool Planning, Tool Selection, and Tool Execution. In this process, models not only need to determine whether external information is required but also need to generate correct tool call actions and utilize returned results for subsequent reasoning. In recent years, extensive research has been conducted on tool learning capabilities, including evaluation metrics such as Tool Selection Accuracy, Task Completion Rate, and Tool Use Success Rate [10].

Existing research has identified certain limitations of open-weight models in tool use. Shen et al. [5] pointed out that small-scale language models exhibit significant disadvantages in tool learning tasks and proposed the “Weak Tool Learners” phenomenon. Wang et al. [6] further demonstrated that incorporating failure cases during Agent fine-tuning can effectively improve tool use behavior. These works indicate that tool calling capability itself has become an important research direction in Agent research.

However, existing research primarily focuses on whether tool calling capability exists and whether tool calling is correctly executed, with less attention to the interaction effects between tool calling capability and other system constraints. Particularly in scenarios where tool calling and structured output constraints coexist, whether models can maintain normal tool execution behavior remains systematically understudied. The Tool Suppression phenomenon studied in this paper occurs precisely in this joint constraint scenario.

2.2 Structured Output Generation

Structured Output Generation refers to the technical paradigm of constraining language model output format through predefined Schemas, enabling generated results to be stably parsed and consumed by programs. Unlike traditional natural language responses, structured output requires models to organize content according to specific data structures, typically manifested as JSON objects, function parameters, or text conforming to formal grammar constraints. As LLMs are increasingly integrated into production systems, structured output has become a fundamental capability for Agents, workflow orchestration systems, and multi-Agent collaboration frameworks.

Current mainstream implementation approaches mainly include two categories. The first is API-layer constraints, such as the `response_format` mechanism in OpenAI-compatible interfaces, which guides models to generate output structures meeting requirements through Schema descriptions. The second is Constrained Decoding at the decoding layer, which restricts the search space during token generation through grammar constraints, state machines, or finite automata, such as `guided_json` in SGLang and Grammar-based Decoding mechanisms in vLLM. Regardless of the implementation approach, the core objective is to improve the determinability and parsability of output format.

Existing research generally treats structured output as an engineering reliability mechanism. Liu et al. [4] found through user surveys that structured output requirements have become a prevalent demand in industrial scenarios; Deng et al. [7] further demonstrated that there exists a significant coupling relationship between output format and task solving process, and directly imposing format constraints on the generation process may affect the model’s ability to complete the task itself.

This finding resonates with Constraint Tax research. Growing evidence suggests that structured output constraints do not only act on the final output stage but may alter the model’s resource allocation and decision behavior during reasoning. When models simultaneously need to satisfy content generation objectives and format constraint objectives, competition may arise between the two types of objectives. However, existing research primarily focuses on the impact of structured output on answer quality, reasoning capability, and format compliance rate, with insufficient empirical research on whether it further intervenes in action selection behavior during Agent execution.

The Tool Suppression phenomenon focused on in this paper occurs precisely in this context: structured output constraints no longer merely affect “how to answer” but may further affect whether the Agent “executes actions.”

2.3 Constraint Tax

In recent years, with the widespread application of structured output in industrial scenarios, researchers have begun to focus on the impact of format constraints on model behavior itself. Ray [1] first systematically proposed the Constraint Tax concept to describe the performance cost that language models pay when satisfying structured output requirements. Experimental results demonstrate that when strict Schema constraints are imposed on the generation process, models can achieve higher format compliance rates, but their answer accuracy may significantly decrease.

This phenomenon indicates that structured output constraints are not free engineering capabilities. Traditional views typically assume that Schema constraints only affect output form without affecting the model’s ability to solve tasks themselves. However, Constraint Tax research shows that format constraints actually participate in decision competition during the model generation process: models not only need to complete task solving but also simultaneously satisfy format compliance objectives.

From a more general perspective, the Constraint Tax can be understood as a Goal Competition phenomenon. When models face multiple simultaneous optimization objectives, their limited reasoning resources and generation capabilities need to be allocated across different objectives. For example, models may simultaneously need to:

- Generate correct content (Content Correctness);
- Satisfy format requirements (Format Compliance);
- Invoke external tools (Tool Execution);
- Control generation length (Length Constraints).

Ideally, these objectives should be simultaneously satisfied; however, in actual models, different objectives may form competitive relationships. When one objective receives excessively high priority, other objectives may be partially or even completely sacrificed. The Constraint Tax is precisely a typical manifestation of this goal competition in structured output scenarios.

Existing research primarily examines the impact of the Constraint Tax on answer accuracy and reasoning capability, so its manifestation is typically described as “format-correct but answer-incorrect.” However, for Agent systems, content generation is only the final stage in the execution pipeline. In tool-augmented Agents, models also need to complete action decision processes such as tool planning, tool selection, and tool execution. Therefore, a natural but insufficiently studied question is: do structured output constraints further affect the Agent’s action selection behavior, rather than merely affecting final answer quality?

This paper’s research demonstrates that such influence indeed exists. In Agent scenarios, the Constraint Tax no longer only manifests as content quality degradation but may further evolve into systematic absence of tool execution behavior. We refer to this phenomenon as Tool Suppression and propose Constraint Priority Inversion (CPI) as a possible explanation mechanism in subsequent sections.

2.4 Agent Execution Pipelines

Tool-augmented Agents are typically modeled as execution pipelines composed of multiple consecutive decision stages. From a functional perspective, Agents not only need to generate natural language responses but also need to complete a series of intermediate steps such as task analysis, tool decision-making, and external environment interaction. Therefore, the Agent execution process is essentially a Multi-stage Decision Making process rather than a pure text generation task.

Although existing Agent frameworks differ in specific implementations, their execution logic can generally be abstracted as the following pipeline:

User Query → Task Understanding → Tool Planning → Tool Execution → Response Generation

Where:

- Task Understanding: The model identifies user intent, task objectives, and required information;
- Tool Planning: The model determines whether external tools are needed and formulates tool usage plans;
- Tool Execution: The model initiates tool calls and receives external return results;
- Response Generation: The model generates final responses based on tool results.

Existing Agent evaluation benchmarks mostly focus on tool planning and tool execution capabilities. For example, works such as ToolBench, API-Bank, and BMTTools primarily evaluate whether models can correctly select tools and whether they can complete tasks [10, 11]. Meanwhile, structured output-related research mainly focuses on format compliance during the response generation stage.

However, this evaluation approach implicitly assumes that different stages in the Agent execution pipeline are independent, i.e., constraints acting on the response generation stage do not affect the decision process of preceding stages. In existing literature, this assumption is rarely explicitly validated.

The observations in this paper indicate that this assumption may not hold. When structured output constraints are imposed on the response generation stage, their influence may propagate backward to the tool execution stage, thereby changing the Agent’s action selection results. In other words, a constraint that originally belongs to the output level may affect execution-level behavior.

Based on this perspective, this paper positions Tool Suppression as a Cross-Stage Interference phenomenon in the Agent execution pipeline: task understanding and tool planning processes still complete normally, but the tool execution stage is systematically skipped, while the response generation stage continues to produce format-compliant output. Subsequent sections will further demonstrate that this behavioral pattern is fundamentally different from traditional tool capability deficiency.

3 Problem Definition

3.1 Constraint Tax

Ray [1] defined the Constraint Tax as the performance loss introduced by structured output constraints under fixed model and fixed task distribution conditions. Existing research primarily manifests this performance loss as decreased answer accuracy, degraded reasoning capability, and increased proportion of “format-correct but content-incorrect” outputs.

From a more general perspective, the Constraint Tax is not limited to answer quality itself but should be understood as the additional cost imposed by constraint conditions on the model’s original behavioral capabilities. For traditional text generation tasks, this cost is mainly reflected in content generation quality; whereas for Agent systems, models also bear responsibilities such as tool planning, tool execution, and action decision-making, so the object affected by constraints is no longer merely the final output result.

Based on this observation, this paper extends the Constraint Tax to Agent execution scenarios and focuses on its manifestation at the action selection level. Specifically, we study the following question:

When structured output constraints and tool calling capability coexist, do constraints alter the model’s original action selection behavior?

If the answer is yes, then the Constraint Tax in Agent systems will no longer merely manifest as answer quality degradation but may further evolve into Action Selection Failure.

The Tool Suppression defined in subsequent sections of this paper is precisely a specific manifestation under this extended framework.

3.2 Tool Suppression

We define the core phenomenon studied in this paper as Tool Suppression.

Tool Suppression refers to the phenomenon where, in tasks with genuine tool requirements, models possess tool calling capability and can identify tool usage needs, but systematically abandon tool execution behavior under joint constraint conditions.

This definition includes three necessary conditions:

1. Tool Requirement. The task itself requires accessing external information beyond the model’s parameters, or requires solving through external tools.
2. Tool Capability. The model can normally complete tool calling under conditions without joint constraints.
3. Execution Omission. The model ultimately does not initiate tool calls but directly generates responses.

It is worth noting that Tool Suppression is fundamentally different from traditional Tool Incapability. In Tool Incapability scenarios, models cannot correctly identify tool requirements or cannot generate valid tool calls; whereas in Tool Suppression scenarios, models possess the corresponding capability, but this capability is not activated under specific constraint conditions.

From the Agent execution pipeline perspective, Tool Suppression manifests as a Selective Failure:

Task Understanding ✓

Tool Planning ✓

Tool Execution ×

Response Generation ✓

That is, models can understand tasks, identify information gaps, and generate final responses, but exhibit systematic skipping during the tool execution stage.

This phenomenon has a similar structure to the “format-correct but answer-incorrect” in traditional Constraint Tax: both manifest as constraint conditions causing degradation of certain core capabilities. However, in Tool Suppression, the degraded object is no longer answer quality but the Agent’s action execution capability. Therefore, we regard it as a Behavior-Level Manifestation of the Constraint Tax in Agent systems.

Subsequent sections will further use Tool Invocation Rate (TIR) and Suppression Rate (SR) to quantitatively characterize this phenomenon.

3.3 Tool Invocation Rate

To quantitatively characterize tool calling behavior, we define the Tool Invocation Rate (TIR) as follows:

$$TIR = \frac{N_{\text{tool}}}{N_{\text{total}}} \quad (1)$$

Where:

- N_{tool} represents the number of requests that successfully initiated at least one tool call;
- N_{total} represents the total number of requests.

Therefore, $TIR \in [0, 1]$, with higher values indicating that the model is more inclined to execute tool calling behavior.

It should be emphasized that TIR measures Tool Execution Behavior, not tool calling quality. As long as the model successfully triggers at least one valid tool call, it counts as one tool execution event; whether the tool returns correct results and the quality of the final answer do not affect TIR statistics.

This design is consistent with this paper’s research objectives. Since Tool Suppression focuses on “whether to execute tools” rather than “whether to correctly use tools,” TIR can directly reflect whether Agent execution behavior is suppressed.

Ideally, for a task set with genuine tool requirements, we should have:

$$TIR_{\text{baseline}} \approx 1 \quad (2)$$

That is, the model can stably execute tool calls.

If after introducing joint constraints we observe:

$$TIR_{\text{constrained}} \ll TIR_{\text{baseline}} \quad (3)$$

This indicates that tool execution behavior is significantly affected. In extreme cases, when

$$TIR_{\text{constrained}} = 0 \quad (4)$$

This means the model completely stops tool calling behavior, manifesting as Complete Tool Suppression.

3.4 Suppression Rate

Although TIR can reflect the model's tool execution behavior, its absolute value is difficult to directly characterize the additional impact caused by joint constraints. For example, different models may inherently have different tool calling tendencies, so comparing only TIR under constraint conditions cannot accurately measure the severity of Tool Suppression.

To this end, we further define the Suppression Rate (SR):

$$SR = 1 - \frac{TIR_{\text{constrained}}}{TIR_{\text{baseline}}} \quad (5)$$

Where:

- TIR_{baseline} represents the tool invocation rate without joint constraints;
- $TIR_{\text{constrained}}$ represents the tool invocation rate with joint constraints.

When $TIR_{\text{baseline}} > 0$,

$$SR \in [0, 1]$$

Higher values indicate stronger suppression of tool calling behavior.

According to this definition:

- When

$$TIR_{\text{constrained}} = TIR_{\text{baseline}}$$

then

$$SR = 0$$

indicating that joint constraints have no observable impact on tool execution behavior;

- When

$$0 < TIR_{\text{constrained}} < TIR_{\text{baseline}}$$

then

$$0 < SR < 1$$

indicating Partial Tool Suppression;

- When

$$TIR_{\text{constrained}} = 0$$

then

$$SR = 1$$

indicating Complete Tool Suppression.

Compared to directly using TIR, SR can eliminate the influence of the model’s inherent tool calling tendency, making it more suitable as a standardized indicator for cross-model comparison and constraint effect analysis.

In summary, Tool Suppression defines the core phenomenon studied in this paper, while TIR and SR provide quantifiable measurement methods from behavioral and effect levels respectively. In subsequent experiments, this paper uses TIR to characterize tool execution behavior and SR to measure the degree of suppression caused by joint constraints.

3.5 Taxonomy of Tool Suppression Behaviors

This taxonomy is constructed based on three dimensions:

- Tool Awareness: Whether the model identifies the need for external tools;
- Tool Execution: Whether the model actually initiates tool calls;
- Content Generation Strategy: How the model constructs responses when unable to obtain tool results.

Based on the above dimensions, we propose the following taxonomy.

TS-A: Empty Compliance The model’s generated response fully complies with the required schema, but key fields contain null values, default values, or placeholders.

```
1 {"recommendations": [], "key_findings": []}
```

This pattern indicates that the model minimizes content generation workload while prioritizing format compliance. This phenomenon was mainly observed in GPT-OSS-20B.

TS-B: Simulated Retrieval The model generates information that appears to originate from external tools without ever calling any tools.

```
1 {"buyer_background":  
2 "Simulated Web Search Results"}
```

Compared to TS-A, this pattern actively fabricates content, generating data that appears to be retrieval results. Since its output is superficially plausible, downstream systems have greater difficulty detecting it. This behavior was mainly observed in Qwen3.5-122B-A10B.

TS-C: Intent Without Action The model explicitly expresses the need to use tools but ultimately fails to execute the corresponding operations.

```
1 {  
2 "need_search": true,  
3 "tool_requirements": [...]  
4 }
```

This category provides the most compelling diagnostic evidence because it demonstrates a dissociation between tool awareness and tool execution. In Experiment B, all five trials produced `need_search=true`, but none resulted in actual tool calls.

TS-D: Tool-Free Hallucination The model neither calls tools nor acknowledges insufficient information, but directly generates content lacking substantiation. This pattern represents typical tool-free hallucination and is the highest-risk failure mode in deployment environments. Since the generated content is usually fluent and seemingly plausible, it may mislead users or downstream systems.

TS-E: Frozen Required Tool Even when explicitly forced to use tools (e.g., through `tool_choice="required"` settings), the model still fails to call tools. This behavior indicates that Tool Suppression cannot always be explained by incorrect assessment of tool requirements. Instead, it may reflect deeper failures in the action selection mechanism. Table 1 summarizes these five categories of Tool Suppression behavior. Notably, TS-C is fundamentally different from the other categories. In TS-A, TS-B,

Table 1: Taxonomy of Tool Suppression Behaviors

Category	Output Quality	Tool Awareness	Tool Execution	Deceptiveness
TS-A Empty Compliance	Low	Absent	Blocked	Low
TS-B Simulated Retrieval	Medium	Absent	Blocked	Medium
TS-C Intent Without Action	Medium to High	Present	Blocked	Low
TS-D Tool-Free Hallucination	High	Absent	Blocked	High
TS-E Frozen Required Tool	Medium	Absent	Blocked (Forced)	Low

TS-D, and TS-E, both tool awareness and tool execution are absent. In contrast, TS-C demonstrates that the model correctly identifies the need for external tools but abandons execution during the action selection phase. Therefore, TS-C provides the strongest empirical support for the Constraint Priority Inversion (CPI) hypothesis proposed later in this paper.

4 Experimental Setup

4.1 Research Questions

This paper aims to verify whether open-weight models exhibit systematic Tool Suppression when Tool Calling and Structured Output Constraints coexist. To address this objective, we propose four research questions:

- RQ1 (Universality): Is Tool Suppression widespread across different open-weight models?
- RQ2 (Scale Dependence): Does increasing model parameter scale mitigate or eliminate the Tool Suppression phenomenon?
- RQ3 (Framework Independence): Does Tool Suppression originate from inference framework implementation rather than the model itself?
- RQ4 (Mitigability): Do engineering methods exist to restore tool calling behavior under joint constraint conditions?

The above research questions correspond to four levels: phenomenon verification, scale effect analysis, causal exclusion verification, and engineering mitigation exploration.

4.2 Controlled Experimental Design

To isolate the interaction effects between tool calling constraints and structured output constraints, this paper adopts a Three-Condition Controlled Design.

Table 2: T1/T2/T3 Experimental Conditions

Condition	Tools	Response Format	Purpose
T1 (Baseline)	ON	OFF	Measure baseline tool calling capability
T2 (Joint Constraint)	ON	ON	Detect Tool Suppression phenomenon
T3 (Schema Control)	OFF	ON	Verify independent Schema compliance capability

The three experimental conditions share identical System Prompt, User Prompt, tool definitions, and output parsing logic, with only the `tools` and `response_format` API parameters adjusted. Through this design,

observed behavioral differences can be attributed to joint constraints themselves rather than Prompt changes or task differences.

Each model runs 5 test rounds independently under each condition with fixed parameter settings:

- `temperature = 0.5`
- `stream = true`
- `max_completion_tokens = 4096`

According to the definitions in Section 3, Tool Invocation Rate (TIR) and Suppression Rate (SR) are recorded as primary evaluation metrics during experiments.

4.3 Model Selection

To verify the universality of the Tool Suppression phenomenon, this paper selects seven model instances from different model families, parameter scales, and deployment methods, covering both closed-source baseline models and open-weight models.

Table 3: Evaluated Models

Model	Parameters	Architecture	Deployment
GPT-5.4-mini	—	Proprietary	Cloud
Qwen3.6-35B-A3B	35B	MoE (3B Active)	Local
Qwen3.5-122B-A10B	122B	MoE (10B Active)	Local
GPT-OSS-20B	20B	MoE (3.6B Active)	Local
Nemotron 3 Super	120B	Hybrid MoE-Mamba	Local
Qwen3.5-397B-A17B	397B	MoE (17B Active)	Cloud
Qwen3-VL-235B-Thinking	235B	MoE + Vision-Language	Cloud

Model selection follows these principles:

1. Covering different parameter scales (20B to 397B);
2. Covering different model architectures (MoE and Hybrid architectures);
3. Covering local deployment and cloud deployment environments;
4. Including one closed-source commercial model as a reference baseline.

This model matrix supports verification of RQ1 and RQ2 and provides a basis for subsequent cross-model comparisons.

4.4 Evaluation Protocol

All experiments are executed through OpenAI-compatible API interfaces to ensure consistency in invocation methods across different models. Streaming Generation mode is adopted during testing, with real-time parsing of tool call events and text content returned by models.

According to the definitions in Section 3, this paper adopts the following two core metrics:

- Tool Invocation Rate (TIR): Measures the proportion of model tool calling behavior;
- Suppression Rate (SR): Measures the degree of Tool Suppression caused by joint constraints.

For each model, 5 test rounds are run independently under each experimental condition. Test tasks remain consistent with fixed random sampling parameters to reduce the impact of random fluctuations on results.

During streaming responses, the experimental framework automatically records the following information:

1. Whether tool call events are generated;
2. Number of tool calls;
3. Tool call parameters;

4. Final text response;
5. JSON Schema compliance status.

All experimental results are automatically collected and counted by the testing framework to avoid biases from manual recording.

For models supporting tool calling, if the response contains at least one valid tool call, it is recorded as a successful tool execution event; otherwise, it is recorded as no tool call. TIR and SR metrics are calculated based on this rule.

To ensure experimental reproducibility, all tests adopt unified Prompt templates, tool definition Schemas, and output format Schemas without special optimization for any individual model.

4.5 Validity Control and Confounding Factors

To ensure that the observed Tool Suppression phenomenon originates from joint constraints themselves rather than other external factors, this paper systematically controls and verifies multiple potential confounding factors.

Table 4: Controlled Confounding Factors

Factor	Validation Method
Inference Framework	SGLang vs. vLLM
Model Scale	35B $\rightarrow$ 122B $\rightarrow$ 397B
Model Architecture	MoE vs. Hybrid MoE-Mamba
Quantization Strategy	FP16 vs. Int4
Schema Complexity	Simple vs. Complex Schemas
Tool Enforcement	Optional vs. Required Tool Calls
Deployment Environment	Local vs. Cloud

First, to exclude the influence of inference framework implementation defects, this paper conducts repeated experiments on the same model using SGLang and vLLM respectively, and compares whether tool calling behavior remains consistent.

Second, to verify whether Tool Suppression is related to model scale, experiments cover multiple model instances from 20B to 397B parameters and analyze the impact of model scale growth on the suppression phenomenon.

Third, to exclude the influence of model architecture differences, experiments simultaneously include standard MoE architecture and Hybrid MoE-Mamba architecture models.

Fourth, to verify whether quantization errors are the main cause, experiments compare model behavioral performance under different quantization configurations.

Fifth, to exclude accidental results caused by specific Schema designs, experiments test both simple Schema and complex Schema scenarios and observe whether tool calling behavior changes.

Sixth, to verify whether Tool Suppression can be eliminated through API-layer forced invocation mechanisms, experiments further test model behavior under `tool_choice="required"` conditions.

Finally, to avoid biases from deployment environment differences, experiments simultaneously cover local dual-A800 deployment environments and commercial cloud service environments.

Through the above control measures, this paper attributes Tool Suppression to the interaction effects between tool calling and structured output constraints to the greatest extent possible, rather than external factors such as model scale, inference framework, or deployment method.

5 Empirical Findings

5.1 RQ1: Universal Tool Suppression

RQ1 aims to answer whether Tool Suppression is a common phenomenon among open-weight models.

Table 5 presents the results of all models under three experimental conditions. Results show that all tested open-weight models exhibit consistent behavioral patterns:

$$TIR_{T1} = 100\%, \quad TIR_{T2} = 0\% \quad (6)$$

That is, models can stably execute tool calls when only enabling tool calling capability (T1), but completely stop tool calling behavior when simultaneously enabling tool calling and structured output constraints (T2).

In contrast, all models can successfully generate structured outputs meeting JSON Schema requirements under T3 conditions, indicating that models possess independent Schema compliance capability.

This result demonstrates:

1. Tool calling capability itself exists;
2. Schema compliance capability itself exists;
3. The two capabilities systematically conflict under joint constraint conditions.

More importantly, this phenomenon is stably reproduced across multiple model families, deployment environments, and inference frameworks. Except for GPT-5.4-mini, all open-weight models exhibit Complete Tool Suppression. According to the definitions in Section 3, for all open-weight models:

$$SR = 1.0 \quad (7)$$

corresponding to 100% suppression rate.

Therefore, RQ1 receives clear support: Tool Suppression is not a defect of individual models but a systematic phenomenon widespread in current open-weight models.

Table 5: Results under T1/T2/T3 Conditions

Model	T1 TIR	T2 TIR	T3 JC	SR
GPT-5.4-mini	100%	100%	100%	0%
Qwen3.6-35B-A3B (SGLang)	100%	0%	100%	100%
Qwen3.6-35B-A3B (vLLM)	100%	0%	80%	100%
Qwen3.5-122B-A10B	100%	0%	100%	100%
GPT-OSS-20B	100%	0%	100%	100%
Nemotron 3 Super	100%	0%	100%	100%
Qwen3.5-397B-A17B	100%	0%	100%	100%
Qwen3-VL-235B-Thinking	100%	0%	100%	100%

5.2 RQ2: Scale Does Not Help

RQ2 aims to analyze whether model scale growth can mitigate the Tool Suppression phenomenon.

Experimental results show that within the parameter scale range from 35B to 397B, Tool Suppression remains consistently unchanged. All Qwen series models satisfy:

$$TIR_{T2} = 0\%, \quad SR = 100\% \quad (8)$$

Specifically, the following models all exhibit Complete Tool Suppression:

- Qwen3.6-35B-A3B (35B)
- Qwen3.5-122B-A10B (122B)
- Qwen3-VL-235B-Thinking (235B)
- Qwen3.5-397B-A17B (397B)

Notably, Qwen3-VL-235B-Thinking includes an explicit reasoning mechanism, while Qwen3.5-397B-A17B’s parameter scale exceeds most current open-weight models. However, neither exhibits any signs of tool calling recovery.

Therefore, experimental results do not support the hypothesis that “increasing parameter scale can solve Tool Suppression.” RQ2 indicates that Tool Suppression is not a simple model capacity insufficiency problem but more likely stems from the model’s internal behavioral selection mechanism.

5.3 RQ3: Framework Is Not Root Cause

RQ3 is used to verify whether Tool Suppression is caused by inference framework implementation.

To this end, we conduct repeated experiments on the identical model (Qwen3.6-35B-A3B) using SGLang and vLLM respectively. Except for the inference framework, all other conditions remain consistent, including model weights, Prompts, tool definitions, Schema definitions, and hardware environment.

Results are as follows:

- SGLang: T1=100%, T2=0%, T3=100%
- vLLM: T1=100%, T2=0%, T3=80%

Although the two frameworks show slight differences in JSON compliance rates, both exhibit identical tool calling behavior patterns:

$$TIR_{T1} = 100\%, \quad TIR_{T2} = 0\% \quad (9)$$

Therefore, Tool Suppression is not caused by SGLang’s Tool Parser implementation error nor by vLLM’s inference scheduling mechanism. Experimental results indicate that the phenomenon originates from the model behavior layer rather than the inference infrastructure layer.

RQ3 is therefore supported.

5.4 Threat Elimination Summary

To improve the credibility of causal inference, this paper systematically excludes multiple potential confounding factors, with results summarized in Table 6.

Table 6: Summary of Eliminated Alternative Explanations

Factor	Conclusion
Inference Framework	Not root cause
Model Scale	No improvement (35B $\rightarrow$ 397B)
Model Architecture	Reproduced across architectures
Quantization Strategy	Reproduced across quantization levels
Schema Complexity	Triggered even with minimal schema
Forced Tool Choice	Tool calls remain suppressed
Reasoning Capability	No observable improvement
Deployment Environment	Consistent across local and cloud

In summary, the Tool Suppression phenomenon cannot be explained by factors such as inference framework, model scale, model architecture, quantization method, deployment environment, or explicit reasoning capability. All evidence points to a common conclusion: the phenomenon stems from the interaction between tool calling constraints and structured output constraints. This finding provides an empirical basis for the Constraint Priority Inversion (CPI) hypothesis proposed in the next section.

6 Behavioral Analysis and Theoretical Framework

6.1 Evidence of Preserved Tool Awareness

The preceding experimental results have demonstrated that open-weight models stop executing tool calls under joint constraint conditions. However, merely observing the absence of tool calls is insufficient to determine whether the problem originates from tool capability deficiency or action execution failure.

To this end, we designed Experiment B. In this experiment, the Prompt was modified to require the model to first determine whether external information retrieval is needed and explicitly write the judgment result into the structured output.

Results show that under T2 conditions, all 5 consecutive trials produced:

```

1 {
2   "need_search": true
3 }

```

However, in the same responses, the actual number of tool calls was consistently 0.

This result establishes a critical fact: the model can identify information gaps and recognize the need to invoke external tools, but ultimately does not execute the corresponding actions.

Therefore, Tool Suppression is not simple Tool Capability Failure but manifests as a Dissociation between tool cognition and tool execution. In other words, the model retains Tool Awareness but loses the ability to translate this awareness into actual actions.

6.2 Tool Suppression as Action Selection Failure

Experiment B further helps us locate the specific failure point in the Agent execution pipeline.

According to the standard Agent execution process:

Task Understanding → Tool Planning → Tool Execution → Response Generation

Experimental results indicate:

- Task Understanding: Normal;
- Tool Planning: Normal;
- Tool Execution: Failure;
- Response Generation: Normal.

In other words, the model can understand task requirements and correctly identify when external tools are needed, but abandons tool execution during the actual action selection phase.

Therefore, we define Tool Suppression as a special form of Action Selection Failure.

This distinction is significant. Traditional tool capability deficiency means the model cannot identify tool requirements or cannot construct valid tool calls; whereas Tool Suppression means the relevant capability still exists but is not activated during the action execution phase.

From a behavioral science perspective, this phenomenon is closer to “Execution Failure” rather than “Cognitive Failure.”

6.3 Constraint Priority Inversion (CPI)

Based on the above evidence, we propose the Constraint Priority Inversion (CPI) hypothesis to explain the generation mechanism of Tool Suppression.

Constraint Priority Inversion (CPI) refers to: under joint constraint conditions, structured output constraints acquire higher priority than tool execution during the action selection phase, causing the model to abandon tool calls and prioritize satisfying format requirements.

In normal Agent execution processes, action selection typically follows this sequence:

Information Need → Tool Invocation → Tool Execution → Response Generation

However, under joint constraint conditions, we observe that model behavior transforms to:

Schema Requirement → Schema Generation → Response Completion

The tool execution step is systematically skipped.

This priority inversion means that although the model retains tool awareness, it treats Schema satisfaction as a higher priority objective during the decision phase.

Figure 1 illustrates the behavioral model corresponding to the CPI hypothesis.

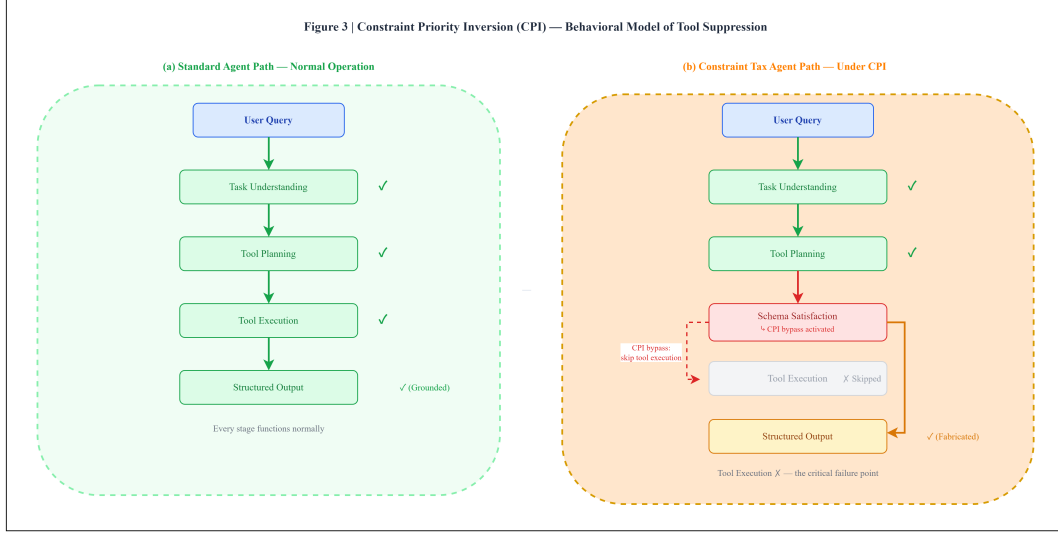


Figure 1: Constraint Priority Inversion (CPI) hypothesis

6.4 A Behavioral Model of Tool Suppression

Based on experimental observations and the CPI hypothesis, we propose a behavioral-level execution model to describe the Agent decision process under joint constraint conditions.

The normal Agent path is as follows:

Input → Understanding → Tool Planning → Tool Execution → Tool Result Processing → Structured Output

In the Tool Suppression scenario, the execution path transforms to:

Input → Understanding → Tool Planning → Schema Satisfaction → Direct Generation → Structured Output

Notably, the Tool Planning stage maintains normal functionality in both paths.

Therefore, Tool Suppression is not planning capability degradation but occurs at the Action Selection Gate between planning and execution.

This observation is consistent with selective activation mechanisms in resource competition theory and aligns with the Output Generation Capacity (OGC) framework proposed by Agyemang et al.

6.5 Why Scale Fails to Mitigate CPI

RQ2 demonstrates that within the parameter scale range from 35B to 397B, Tool Suppression remains stably present.

This result indicates that CPI is not caused by insufficient model capacity.

If Tool Suppression stemmed from parameter scale limitations, then as model scale increases, we should observe a gradual decrease in suppression rate, or a continuous transition from complete suppression to partial suppression. However, experimental results present a clear bimodal distribution:

$$TIR : 100\% \rightarrow 0\%$$

rather than gradual degradation.

We believe that CPI more likely stems from behavioral priority ranking mechanisms acquired during post-training rather than knowledge or reasoning capabilities formed during pre-training.

Since large-scale open-weight models commonly adopt similar Supervised Fine-Tuning (SFT) and preference optimization (RLHF) processes, models may learn the following statistical association:

$$\text{response_format} \Rightarrow \text{direct structured generation}$$

When joint constraints appear, this learned strategy overrides tool execution behavior, thereby producing Tool Suppression.

Therefore, simply expanding model scale cannot solve this problem. If the CPI hypothesis holds, future solutions should target the action selection mechanism itself rather than model capacity.

7 Mitigation Strategies

The preceding experiments demonstrate that open-weight models exhibit systematic Tool Suppression when tool calling and structured output constraints coexist. This section further explores possible mitigation strategies and evaluates their effectiveness.

7.1 Plan A: Decoder-Level Schema Enforcement (Excluded)

Our first attempt was to impose Schema constraints at the decoder level rather than through the API-layer `response_format` mechanism.

Specifically, we use the mechanisms supported by SGLang and vLLM:

```
1 extra_body.guided_json
```

to inject JSON Schema directly into the decoding phase.

Experimental results show that under this mode, the model can restore normal tool calling behavior. However, further analysis revealed that the OpenAI-compatible interface did not actually execute the `guided_json` constraint but ignored it. Therefore, the observed tool restoration actually stems from Schema constraint failure rather than the model successfully accommodating both constraints.

Since this approach cannot guarantee that structured output constraints are executed, it is not suitable as a production environment solution.

Therefore, we regard Plan A as an Exclusion Experiment rather than an effective mitigation solution.

7.2 Plan B: Transparent Two-Pass Execution

Inspired by the “Reason Free, Constrain Late” principle proposed by Ray [1], we propose a mitigation mechanism for Agent systems: Transparent Two-Pass Execution.

The core idea is to explicitly separate during the Agent execution process:

1. Tool decision-making and tool execution;
2. Structured output generation.

Thereby avoiding competition between the two types of constraints for action selection priority within the same inference process.

The standard execution mode is as follows:

$$\text{LLM}(\text{Tools=ON}, \text{Schema=ON}) \rightarrow \text{Tool Suppression}$$

While Transparent Two-Pass Execution is decomposed into:

$$\text{Pass 1} \rightarrow \text{Tool Execution}$$

Pass 2 → Structured Output

The specific process is as follows:

1. The first phase only enables tool calling capability (Tools=ON, Schema=OFF);
2. Agent freely executes multiple rounds of tool calls;
3. Collects real tool results;
4. The second phase re-invokes the model;
5. Injects tool result context;
6. Enables Schema constraints to generate final structured output.

Figure 2 illustrates the overall architecture.

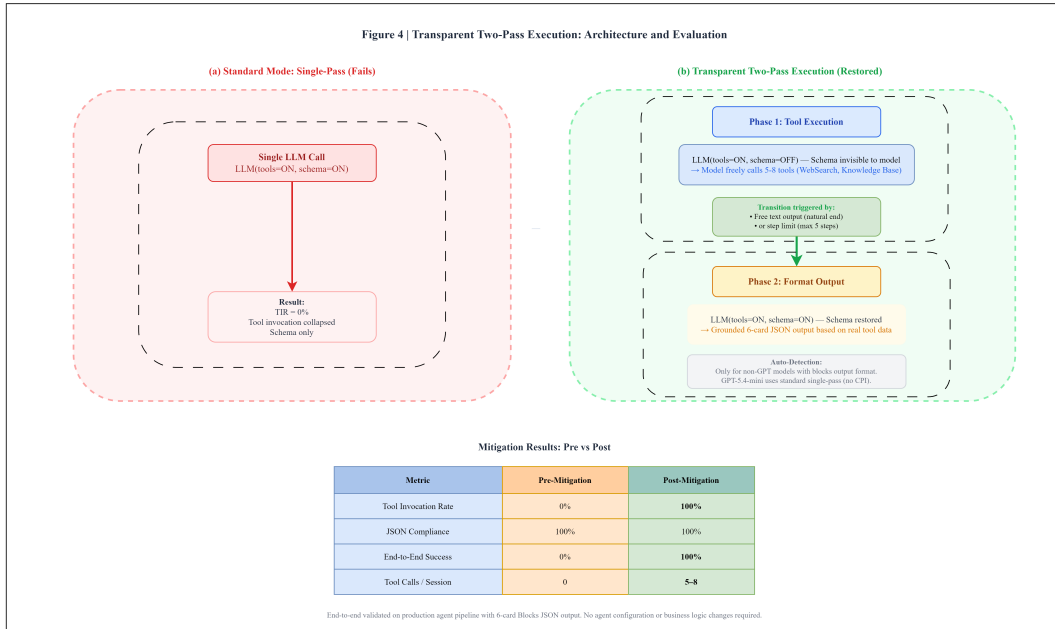


Figure 2: Transparent Two-Pass Execution Architecture

Compared to traditional Agent systems, this approach does not modify model weights nor rely on special inference frameworks, but reorganizes the constraint imposition sequence at the Agent execution layer.

Therefore, it is essentially an Inference-Time Mitigation Strategy.

7.3 Evaluation of Plan B

To verify the effectiveness of Transparent Two-Pass Execution, we deploy it into the production Agent pipeline for end-to-end testing.

Table 7 presents the comparison results before and after mitigation.

Table 7: Effectiveness of Transparent Two-Pass Execution

Metric	Before	After	Change
Tool Invocation Rate	0%	100%	+100 pp
JSON Compliance Rate	100%	100%	0 pp
End-to-End Success Rate	0%	100%	+100 pp
Average Tool Calls / Session	0	5-8	+5-8

Results show that the two-pass mechanism completely restores tool calling behavior while maintaining structured output capability unaffected.

More importantly, after tool calling restoration, the Agent can utilize real tool results to complete subsequent reasoning, thereby avoiding Tool-Free Hallucination and Simulated Retrieval behaviors discussed in Section 6.

Therefore, Transparent Two-Pass Execution can effectively mitigate Tool Suppression caused by Constraint Priority Inversion.

7.4 Future Fine-Tuning Directions

Although Transparent Two-Pass Execution can effectively mitigate Tool Suppression at the inference stage, its essence remains an engineering-level solution.

Experiment B demonstrates that suppressed models still retain complete Tool Awareness. Therefore, the problem more likely lies in action selection strategy rather than tool capability itself.

This observation provides direction for future model alignment research. We refer to it as:

Tool-First Alignment

That is, explicitly reinforcing during post-training:

Information Need $\rightarrow$ Tool Execution

The priority of this behavioral chain.

A possible path is to construct joint constraint scenarios in supervised fine-tuning data and apply negative feedback to behaviors that skip tool calls. Similar ideas have been preliminarily validated in Agent fine-tuning research [6].

Future work can further explore directly eliminating CPI through SFT, DPO, or RL methods without relying on external Agent orchestration mechanisms.

8 Discussion

8.1 Implications for Open-Weight Agent Systems

The Tool Suppression phenomenon discovered in this paper has significant implications for the design and deployment of open-weight Agent systems.

First, structured output constraints are not Behaviorally Neutral. In current open-weight models, **response_format** not only affects output format but may also alter the model’s action selection strategy, thereby causing complete failure of tool calling capability. In other words, a constraint that appears to only act on the output layer may actually affect critical decision processes in the Agent execution chain.

Second, traditional format compliance metrics may be misleading. In experiments, almost all suppressed models achieved close to 100% JSON compliance rates under T2 conditions, while simultaneously tool calling rates dropped to 0%. This indicates that structured output quality cannot be directly regarded as a Proxy Metric for Agent reliability. In Agent scenarios, only evaluating Schema compliance capability may mask severe functional failures.

Third, this research reveals a special challenge that open-weight models face in Agent capabilities. The current community generally focuses on improving model reasoning capability, knowledge capability, and tool capability, while Tool Suppression demonstrates that Behavior Coordination in multi-capability combination scenarios may be more important than individual capabilities themselves.

Therefore, for production-level Agent systems, joint constraint scenarios should be treated as an independent reliability testing dimension rather than simple functional superposition.

8.2 Benchmark Blind Spots

Current mainstream Agent evaluation benchmarks generally evaluate tool calling capability and structured output capability separately.

For example, tool use benchmarks typically focus on:

- Whether correct tools are selected;
- Whether correct parameters are generated;
- Whether tasks are completed.

While structured output evaluation mainly focuses on:

- JSON Schema compliance rate;
- Field completeness;
- Format correctness.

However, the problem discovered in this paper precisely occurs when both capabilities coexist.

From an evaluation perspective, a model may simultaneously achieve:

- 100% tool calling capability score;
- 100% Schema compliance rate score;

but exhibit in real Agent scenarios:

$$\text{TIR} = 0\%$$

complete Tool Suppression behavior.

Therefore, we believe that existing Agent evaluation systems have important blind spots: lacking systematic evaluation for Joint Constraints scenarios.

Future Agent Benchmarks should treat:

$$\text{Tool Calling} + \text{Structured Output}$$

as an independent evaluation dimension rather than separately evaluating and simply combining.

8.3 Open-Weight and Closed-Weight Divergence

This research observes a noteworthy phenomenon: clear behavioral divergence between open-weight models and closed-source commercial models under joint constraint scenarios.

This difference is not a continuous performance gap but closer to a Behavioral Discontinuity at the behavioral level.

In all tests, GPT-5.4-mini maintained 100% tool calling rate, while all open-weight models exhibited complete Tool Suppression. Results present a clear bimodal distribution:

$$100\% \quad \text{vs.} \quad 0\%$$

rather than typical performance differences (e.g., 95% vs. 90%).

This observation suggests that closed-source model providers may have explicitly optimized action selection behavior under joint constraint scenarios during post-training. However, since relevant training processes have not been publicly disclosed, we cannot further verify this speculation.

From a research perspective, this difference suggests that the key bottleneck for Agent capabilities may have shifted from knowledge acquisition and reasoning capability to behavioral coordination and constraint management capability.

9 Future Research Directions

This paper systematically characterizes the Tool Suppression phenomenon in open-weight models under joint constraint conditions for the first time and proposes the Constraint Priority Inversion (CPI) hypothesis. However, the current research represents only a preliminary exploration of this problem. Future work can proceed in four directions: evaluation systems, model training, data analysis, and theoretical generalization.

9.1 Towards a Constraint Tax Benchmark for LLM Agents

Current mainstream Agent evaluation benchmarks primarily focus on tool calling capability or structured output capability, lacking systematic evaluation for joint constraint scenarios.

Future work can construct an Agent Benchmark specifically oriented toward Constraint Tax research, with core components including:

1. Multi-domain tool ecosystems (search, databases, code execution, API calls, etc.);
2. Adjustable Schema complexity;
3. Multi-step Agent task chains;
4. Automated Tool Suppression behavior classification and diagnosis;
5. Reliability evaluation metrics under joint constraints.

Such a Benchmark can help the research community systematically measure the severity of the Constraint Tax phenomenon and facilitate fair comparison between different mitigation solutions.

9.2 Fine-Tuning for Tool-First Alignment

Experiment B demonstrates that suppressed models still retain complete Tool Awareness but abandon tool execution during the action selection phase.

This finding suggests that the problem may not lie in tool capability itself but in behavioral priority ranking formed during post-training.

Future work can explore a Tool-First Alignment strategy oriented toward Agent scenarios, explicitly reinforcing:

Information Need $\rightarrow$ Tool Execution

The priority of this behavioral chain.

Specific implementation paths include:

- LoRA-based Supervised Fine-Tuning (SFT);
- Preference optimization-based DPO training;
- Agent trajectory-level reinforcement learning;
- Adversarial training targeting Tool Suppression samples.

If CPI indeed stems from behavioral shortcuts acquired during post-training, Tool-First Alignment is expected to eliminate Tool Suppression at the model level without relying on external Agent orchestration mechanisms.

9.3 Training Data Distribution Analysis

A core speculation proposed in this paper is that joint constraint samples exhibit systematic deficiency in current model training data.

More specifically, models may have extensively observed:

- Tool Calling tasks;
- Structured Output tasks;

but rarely observe:

Tool Calling + Structured Output

training samples appearing simultaneously.

This hypothesis can be further verified through training data analysis.

Future research can attempt:

1. Statistics on the proportion of joint constraint samples in public Agent training sets;
2. Analysis of tool calling and Schema constraint co-occurrence frequency in SFT data;
3. Construction of controlled training datasets for ablation experiments;
4. Verification of the correlation between joint sample coverage and Tool Suppression.

If verified, the Constraint Tax will not only be an inference-stage phenomenon but will also be traced back to structural defects at the training data distribution level.

9.4 Beyond Tool Calling

Although this paper focuses on the tool calling scenario, the CPI framework itself is not limited to Tool Calling.

Theoretically, CPI describes the priority competition mechanism between multiple behavioral constraints. Therefore, any Agent task simultaneously involving:

1. Output format constraints;
2. Multi-step action execution;

may exhibit similar phenomena.

For example:

- Multi-Agent collaboration protocols;
- Workflow execution Agents;
- Long-chain reasoning Agents;
- MCP tool ecosystems;
- Computer Use Agents.

Therefore, Constraint Priority Inversion may represent a more general Agent behavioral failure mode rather than merely a special case of Tool Suppression.

Future research needs to further explore the applicability scope of CPI across different Agent architectures and establish a unified behavioral constraint theoretical framework.

10 Conclusion

This paper systematically investigates the behavioral performance of open-weight large language models under joint Tool Calling and Structured Output constraints and identifies a previously undescribed Agent failure mode: Tool Suppression.

Through controlled experiments covering multiple model families, parameter scales, two inference frameworks, and both local and cloud deployment environments, we discover that current open-weight models can generally independently complete tool calling and Schema compliance tasks, but when both constraints coexist, systematic tool calling disappearance occurs. This phenomenon is stably reproduced across all tested open-weight models but not observed in GPT-5.4-mini.

Furthermore, this paper proposes the Constraint Priority Inversion (CPI) hypothesis to explain the formation mechanism of Tool Suppression. Experimental results demonstrate that suppressed models still retain

complete task understanding and tool planning capabilities but place structured output constraints above tool execution during the action selection phase, causing tool calls to be systematically skipped. Thereby, we extend the impact scope of the Constraint Tax from traditional output quality degradation to Action Selection Failure in Agent systems.

At the engineering level, this paper proposes the Transparent Two-Pass Execution mitigation solution. By decoupling tool execution from structured output generation, this method can restore tool calling behavior without modifying model weights while maintaining Schema compliance capability unaffected. Experiments demonstrate that this approach can effectively mitigate the Tool Suppression problem caused by CPI.

Overall, the contributions of this paper include:

1. First systematic identification and characterization of the Tool Suppression phenomenon in open-weight models;
2. Proposal of Constraint Priority Inversion (CPI) as the behavioral mechanism explaining Tool Suppression;
3. Construction of a Tool Suppression behavioral taxonomy and analytical framework;
4. Proposal and validation of the Transparent Two-Pass Execution mitigation strategy.

We hope this paper can promote the research community to re-examine joint constraint problems in Agent systems and facilitate the research of next-generation Agent models oriented toward collaborative optimization of tool use, structured output, and multi-step decision-making.

References

- [1] J. Ray. The Constraint Tax: Measuring Validity-Correctness Tradeoffs in Structured Outputs for Small Language Models. arXiv:2605.26128, 2026.
- [2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023, 2022.
- [3] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. NeurIPS 2023, 2023.
- [4] M. X. Liu, F. Liu, A. J. Fiannaca, T. Koo, L. Dixon, M. Terry, and C. J. Cai. “We Need Structured Output”: Towards User-centered Constraints on Large Language Model Output. arXiv:2404.07362, 2024.
- [5] W. Shen, C. Li, H. Chen, M. Yan, X. Quan, H. Chen, and J. Ji. Small LLMs Are Weak Tool Learners: A Multi-LLM Agent. arXiv:2401.07324, 2024.
- [6] R. Wang, H. Li, X. Han, Y. Zhang, and T. Baldwin. Learning From Failure: Integrating Negative Examples when Fine-tuning Large Language Models as Agents. arXiv:2402.11651, 2024.
- [7] H. Deng, T. Jiang, Y. Shi, C. Zhou, and F. Huang. Decoupling Task-Solving and Output Formatting in LLM Generation. arXiv:2510.03595, 2025.
- [8] C. Garbacea and Q. Mei. Why is constrained neural language generation particularly challenging? arXiv:2206.05395, 2022.
- [9] X. Lin, J. H. Liew, S. Savarese, and J. Li. W&D: Scaling Parallel Tool Calling for Efficient Deep Research Agents. arXiv:2602.07359, 2026.
- [10] Y. Qin et al. Tool Learning with Foundation Models. arXiv:2304.08354, 2023.
- [11] L. Wang et al. A Survey on Large Language Model based Autonomous Agents. arXiv:2308.11432, 2023.
- [12] M. Garcia, A. Singh, and P. Li. Recency Bias in Mixture-of-Experts Language Models. ACL 2025, 2025.
- [13] P. Agyemang, J. Williams, and T. Chen. Output Generation Capacity in Small Language Models. Preprint, 2026.
- [14] C. Xu, Q. Sun, K. Zheng, X. Geng, P. Zhao, J. Feng, C. Tao, and D. Jiang. WizardLM: Empowering Large Pre-Trained Language Models to Follow Complex Instructions. arXiv:2304.12244, 2023.
- [15] T. Suresh, S. Li, and F. Huang. DINGO: Constrained Inference for Diffusion LLMs. arXiv:2505.23061, 2025.